

LémanFlow – Rapport produit & technique complet

👤 Author	S Said Kaci
⚙️ Status	Draft
☰ Category	
🕒 Last edited time	@September 26, 2025 10:59 PM



1. Executive Summary

Le problème des hackathons aujourd'hui

Les hackathons sont devenus un levier incontournable d'innovation et de recrutement. Pourtant, la distribution des récompenses (grants, attestations, prix

de participation) reste lourde, opaque et peu inclusive : gestion manuelle, délais de paiement, complexité d'accès pour les non-initiés au Web3 (wallets, gas, seed phrases). Les sponsors et organisateurs peinent à mesurer l'impact réel et à engager durablement les communautés.

La solution LémanFlow

LémanFlow automatise la distribution de micro-grants et d'attestations lors d'événements tech (hackathons, conférences, festivals) grâce à une plateforme Web3 fluide, sans gas ni wallet, basée sur Sui. Les utilisateurs se connectent via zkLogin (Google, GitHub...), reçoivent un NFT soulbound attestant leur participation, et peuvent réclamer leurs récompenses instantanément, tout cela sans friction technique.

Impact

- **Participants** : onboarding sans wallet, expérience fluide, récompenses immédiates et traçables, preuve de participation on-chain.
- **Organisateurs** : automatisation des distributions, analytics en temps réel, meilleure attractivité pour les sponsors.
- **Sponsors** : visibilité on-chain, traçabilité des grants, analytics d'impact, engagement communautaire renforcé.

2. Architecture technique

Vue d'ensemble

Couche	Stack principale	Rôle
Frontend	Next.js, React	UX fluide, interactions utilisateurs
Backend/API	Fastify, Node.js	Orchestration, QR signer, sponsoring
Smart contracts	Move (Sui blockchain)	Logiciels de grants, attestations, SBT

Schéma de flux (simplifié)

Utilisateur → [Frontend Next.js] → [Backend Fastify] → [Smart Contracts Move/Sui]

↗ (QR scan) ↔ (API QR signer + sponsoring gas)

Technologies utilisées & pourquoi

- **Move (Sui)** : sécurité, modularité, gestion native des objets (NFT/GBT), dynamic fields pour la scalabilité[webpage-22,webpage-23].
- **zkLogin** : authentification Web2 (Google, GitHub...), privacy, onboarding massif sans seed phrase[webpage-4,webpage-5].
- **Gasless sponsoring** : UX sans gas, prise en charge des frais par l'orga/sponsor, onboarding frictionless[webpage-26,webpage-27].
- **ECDSA** : signature QR pour vérification cross-device, sécurité des claims[webpage-24,webpage-25].
- **Dynamic Fields** : gestion dynamique des missions, attestations, et pools de grants[webpage-22,webpage-23].

3. Contrats intelligents (Move)

Structures principales

- **Passport** : NFT soulbound, prouvant la participation à un événement.
- **Mission** : tâche ou challenge à accomplir, associée à une attestation et une récompense.
- **Attestation** : SBT délivré après validation d'une mission (preuve d'action, présence...).
- **GrantPool** : pool de fonds (SUI) à distribuer automatiquement selon les règles définies.

Fonctions clés

- `init_pool` : initialisation d'un pool de grants pour un événement.
- `create_mission` : création d'une mission à accomplir.
- `mint_passport` : mint du NFT soulbound pour chaque participant.

- `claim_attestation` : validation et mint d'une attestation SBT.
- `distribute` : distribution automatisée des micro-grants aux wallets des participants.

Logique de sécurité

- **Soulbound** : NFTs non transférables (SBT), preuve immuable de participation ou d'accomplissement[webpage-10,webpage-14].
- **Timestamps** : horodatage des claims pour éviter les fraudes.
- **Anti-double claim** : chaque mission/attestation ne peut être réclamée qu'une fois par wallet.
- **Tests automatisés** : tests unitaires Move sur mint, claim, distribution, double claim, edge cases[notion-5,artifact,0].

Extrait Move (exemple SBT)

```
struct Attestation has key {
  id: UID,
  mission_id: u64,
  owner: address,
  timestamp: u64,
  // pas d'ability 'store', donc non-transférable (soulbound)
}
```

[webpage-14]

4. Backend & API

Endpoints exposés

- `POST /api/login` : zkLogin, génération de session utilisateur.
- `GET /api/missions` : liste des missions disponibles.
- `POST /api/scan` : scan QR, validation mission, signature ECDSA.
- `POST /api/claim` : claim attestation, sponsoring transaction Move.

- `POST /api/admin/init` : création pool, missions, gestion admin.

Signature des QR en ECDSA

- Génération côté backend d'un challenge unique (nonce, mission_id)
- Signature ECDSA côté serveur (clé privée orga)
- Vérification côté smart contract ou backend que le QR signé correspond au bon utilisateur/mission[webpage-25].

Sponsoring des transactions Move

- Utilisation de la primitive "sponsored transactions" de Sui : l'orga/sponsor paie le gas pour l'utilisateur[webpage-26,webpage-27].
- Flow : backend collecte la transaction → ajoute un objet GasData → signe → soumet au réseau Sui.

Exemple de payload

```
{
  "mission_id": "42",
  "user_zklogin": "0xabc...",
  "qr_signature": "0x123...",
  "action": "claim"
}
```

5. Frontend & UX

Pages principales

- `/login` : onboarding zkLogin (Google, GitHub, etc.)
- `/missions` : liste des missions à accomplir, état des rewards.
- `/scan` : scan QR pour valider une mission sur place.
- `/admin` : dashboard organisateur (création missions, suivi rewards, analytics).

Flow utilisateur

1. **Login** (zkLogin, pas de wallet requis)
2. **Mint passport** (NFT soulbound de participation)
3. **Missions** (liste des challenges à accomplir)
4. **Scan QR** (validation sur place ou en ligne)
5. **Rewards** (attestation SBT + micro-grant distribué automatiquement)

UX fluide sans gas ni wallet

- **Sponsoring** : aucun frais pour l'utilisateur, transactions signées et financées par l'orga.
- **zkLogin** : login OAuth classique, onboarding Web2-like, privacy by design[webpage-4,webpage-5].
- **QR signer** : validation simple, cross-device, sécurité renforcée.

6. Business & Scalabilité

TAM / SAM / SOM hackathon economy

Marché	Taille 2023	Proj. 2031	CAGR
Hackathons (global)	\$1,5B	\$5,1B	15,1%
Hackathon software	\$1,2B (2024)	\$3,5B (2033)	15,5%
[webpage-6,webpage-7]			

- **TAM** : tous les événements tech/hackathons mondiaux.
- **SAM** : hackathons, conférences, festivals tech digitaux.
- **SOM** : early adopters Web3, events partenaires, VCs, universités.

Modèle économique

- **Frais plateforme** : % sur grants distribués, forfaits SaaS pour orga.
- **Analytics premium** : dashboards avancés pour sponsors/partenaires.

- **Fonctionnalités avancées** : votes, badges, custom SBT, API intégration.

Roadmap

Version	Fonctionnalités majeures	Cible
V1	Hackathon grants, SBT, sponsoring, QR signer	Hackathons
V2	Conférences, votes, analytics avancés	Conférences
V3	Intégration API, badges custom, multi-événements	Écosystème élargi
V4	DAO, gouvernance, grants cross-chain	Web3 global

Cas d'usage au-delà des hackathons

- **Conférences** : badge de présence, micro-rewards pour feedback.
- **Festivals** : rewards pour bénévoles, SBT pour artistes.
- **Formations** : attestations de suivi, micro-grants pour projets étudiants.
- **Open innovation** : concours d'idées, bounties automatisés.

7. Livrables & structure du repo

Arborescence des dossiers (exemple)

```

/lemanflow
  /contracts    # smart contracts Move
  /backend      # Fastify API, QR signer, sponsoring
  /frontend     # Next.js, pages utilisateur/admin
  /scripts      # CLI, déploiement, tests automatisés
  /docs         # documentation technique + produit
  /slides       # pitch deck, présentations
  README.md

```

Scripts CLI

- `mint-passport`
- `create-mission`

- `claim-attestation`
- `distribute-grants`
- `run-tests`

README & docs internes

- Installation, setup, guides de déploiement, flows d'intégration.
- FAQ développeur, guides best practices Move/Sui.

Slides pitch deck

- Vision, marché, démo, équipe, projections, business model.

8. Annexe technique

Extraits de code Move

```
public fun mint_passport(user: address) {  
    let passport = Passport { id: object::new(), owner: user, timestamp: clock::now() };  
    // Mint SBT non-transférable  
}
```

Exemple .env

```
SUI_NETWORK=mainnet  
BACKEND_PRIVATE_KEY=...  
SPONSOR_ADDRESS=0x...  
ZKLOGIN_CLIENT_ID=...
```

Test CLI à suivre

```
npx lemanflow-cli test --network devnet
```


Génération de clés sponsor/org

```
openssl ecparam -name secp256k1 -genkey -noout -out sponsor-key.pem  
openssl ec -in sponsor-key.pem -pubout -out sponsor-pub.pem
```

Pour les développeurs

- Move/Sui : exploitez Dynamic Fields pour scaler vos missions et rewards, suivez les best practices Move (sécurité, tests, modularité)[webpage-22,webpage-23].
- Backend Fastify : endpoints RESTful, sponsoring gas, gestion QR signer ECDSA.
- Frontend Next.js : focus UX, intégration zkLogin, flows QR scan natifs.

Pour les business angels

- TAM/SAM/SOM en croissance rapide (CAGR >15%), marché mondial hackathon >\$5B d'ici 2031[webpage-6].
- Modèle scalable, extensible à d'autres verticaux (éducation, RH, open innovation).
- Différenciation : UX sans friction, analytics premium, composabilité Web3.

Pour les partenaires d'écosystème

- Intégration API pour plateformes d'événements, sponsors, universités.
- Customisation SBT, badges, missions selon vos besoins.
- Gouvernance future (DAO), cross-chain, analytics avancés.

LémanFlow, c'est la nouvelle norme du reward événementiel Web3 : simple, automatisée, inclusive.